



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

PROYECTO INTEGRADOR

BASTION

Boveda Personal de Credenciales

El impacto de las nuevas tecnologías en la sociedad

David Alexander Benz Zambrano

Logica de Programacion · Msc. Lilian Aman · Paralelo 1-CIB-1A

Quito - Ecuador · junio de 2026

El problema

Una persona administra hoy decenas de credenciales. Sin herramientas, recurre a practicas inseguras:

1

Reutilizacion

Usar la misma contraseña en muchos servicios: filtrar uno compromete todos.

2

Claves debiles

Contraseñas cortas y predecibles, fáciles de adivinar por fuerza bruta.

3

Sin cifrar

Guardarlas en notas, hojas de cálculo o archivos de texto plano.

La solución: BASTION

Una boveda personal de credenciales de línea de comandos en Python.
Genera contraseñas seguras y las guarda en un archivo local protegido por una única contraseña maestra.

Local, sin nube

Los datos nunca salen del equipo: menor superficie de ataque.

Contraseña maestra

Su hash SHA-256 protege el acceso; la clave nunca se guarda en claro.

Generador seguro

Crea claves robustas con indicador de fortaleza.

Organizada

Credenciales por categorías, búsqueda e historial de accesos.

Arquitectura en tres capas

CAPA 1

Presentacion

Menus de la interfaz CLI e interaccion con el usuario.

CAPA 2

Logica de negocio

Generador, validaciones, bucles y casos de uso.

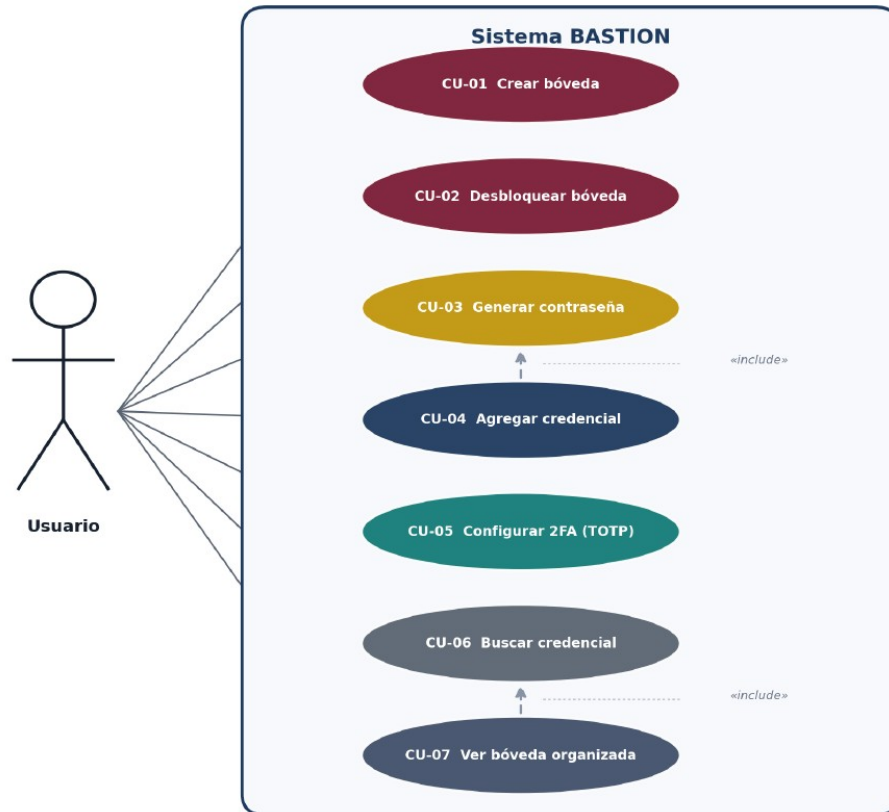
CAPA 3

Persistencia

Lectura y escritura del archivo JSON local.



Funcionalidades del sistema



- CU-01 Crear bóveda
- CU-02 Iniciar sesión (desbloquear)
- CU-03 Generar contraseña segura
- CU-04 Agregar credencial
- CU-05 Configurar 2FA (TOTP) - planificado
- CU-06 Buscar credencial por nombre
- CU-07 Ver todas las credenciales
- Extra Cambiar maestra / Ver historial

Integracion de las cuatro unidades

1

UNIDAD 1

Analisis y diseno

Definicion del problema; diagramas de casos de uso, flujo y arquitectura.

2

UNIDAD 2

Entorno y tipos

GitHub para control de versiones; variables str, int y bool.

3

UNIDAD 3

Bucles y decisiones

while con contador, for con range y .items(), if/elif/else, operadores.

4

UNIDAD 4

Datos y funciones

Tupla CATEGORIAS, lista historial, funciones con parametros por defecto.

El código en acción

Cada estructura de las cuatro unidades aparece de forma explícita y comentada en el código:

```
while intentos < MAX_INTENTOS:                                # bucle con contador (login, 3 intentos)

    for i in range(longitud):                                  # genera la contraseña caracter por caracter

        for s, d in credenciales.items():                     # recorre el diccionario de credenciales

            if longitud >= 12 and tipos >= 3:                  # operadores relacionales y logicos

                CATEGORIAS = ("Redes", "Banca", ...)           # TUPLA de categorias (Unidad 4)

                historial = [] # .append(...)                  # LISTA de accesos con fecha y hora

                def registrar_acceso(b, ev="..."):              # funcion con parametro por defecto
```

Reflexion: impacto en la sociedad

Seguridad al alcance de todos

Los gestores permiten claves unicas y robustas sin memorizarlas: democratizan buenas practicas que antes eran solo de expertos.

Comodidad frente a privacidad

Centralizar todas las claves agiliza, pero concentra el riesgo. El diseno responsable equilibra ambos: BASTION opera local, sin nube.

Etica y datos personales

Custodiar credenciales implica un deber de cuidado. La LOPDP ecuatoriana exige finalidad, minimizacion y seguridad en el tratamiento de datos.

El futuro de la autenticacion

Passkeys (FIDO2) y criptografia post-cuantica marcan la hoja de ruta; los gestores siguen siendo infraestructura de confianza.

Conclusiones y limitaciones

CONCLUSIONES

- Las estructuras de las cuatro unidades resuelven un problema real de gestion de credenciales.
- La arquitectura en tres capas aporta orden, claridad y mantenibilidad.
- Guardar solo el hash de la maestra y operar local refleja buenas practicas de seguridad.
- El proyecto integra programacion y ciberseguridad sobre estandares reconocidos.

LIMITACIONES

- Las credenciales se guardan en JSON sin cifrado AES (planificado con Argon2id).
- Almacenamiento local de un solo usuario, sin acceso concurrente.
- Sin segundo factor de autenticacion aun.
- Interfaz de linea de comandos; la grafica queda como evolucion.

La aplicacion web en vivo

El mismo codigo de bastion.py se ejecuta dentro del navegador con Pyodide. La docente puede entrar, usar la herramienta y ver el codigo Python corriendo por dentro.

Python real

El codigo de bastion.py corre en el navegador mediante Pyodide (WebAssembly); no se reescribe en JavaScript.

Consola en vivo

Un panel muestra cada llamada a Python y su resultado: transparencia total del funcionamiento.

Sin instalar nada

Publicada en GitHub Pages; se abre desde cualquier navegador, en escritorio o movil.

Pruebala en vivo: bluenovasas.github.io/bastion-uide/webapp/

Gracias

BASTION - Boveda Personal de Credenciales

github.com/Bluenovasas/bastion-uide

David Alexander Benz Zambrano · UIDE · 1-CIB-1A · junio de 2026